

Experiment No: 1

Implement the program of substitution and transposition ciphers

Name: Rutuja Dundesh Mane
Div: A

Roll No: A44
PRN: 2324000718

TranspositionClient.java

```
import java.io.*;
import java.net.*;

public class TranspositionClient {

    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 8080);

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(
                socket.getOutputStream(), true);

            // Send request
            out.println("REQUEST_QP");

            int key = Integer.parseInt(in.readLine());
            String encryptedText = in.readLine();

            System.out.println("Encrypted Text: " + encryptedText);

            // Decrypt
            String decryptedText = decrypt(encryptedText, key);

            System.out.println("Decrypted Text: " + decryptedText);
```

```

        socket.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
}

// TRANSPOSITION DECRYPTION
private static String decrypt(String text, int key) {
    int len = text.length();
    int rows = len / key;

    char[][] matrix = new char[rows][key];
    int index = 0;

    // Step 1: Fill matrix column-wise
    for (int j = 0; j < key; j++) {
        for (int i = 0; i < rows; i++) {
            matrix[i][j] = text.charAt(index++);
        }
    }

    // Step 2: Read row-wise
    StringBuilder decrypted = new StringBuilder();
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < key; j++) {
            decrypted.append(matrix[i][j]);
        }
    }

    return decrypted.toString().replace("X", "");
}
}

```

```
import java.awt.*; import
java.io.*; import
java.net.*; import
java.util.HashMap; import
java.util.Map; import
javax.swing.*;

public class ExamServerGUI extends JFrame {

    private ServerSocket serverSocket;
    private JTextArea logArea;

    // Simulated database
    private Map<String, String> qpDatabase;

    public ExamServerGUI() {
        setTitle("Exam Server - Transposition Cipher");
        setSize(600, 400);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        logArea = new JTextArea();
        logArea.setEditable(false);
        add(new JScrollPane(logArea), BorderLayout.CENTER);
```

```
        JButton startBtn = new JButton("Start Server (8080)");
startBtn.addActionListener(e -> startServer());    add(startBtn,
BorderLayout.SOUTH);
```

```
        // Sample Question Papers
qpDatabase = new HashMap<>();
qpDatabase.put("10-CS101-1",
        "Q1. What is Java?\nQ2. Explain OOP concepts.\nQ3. Write features
of Java.");
```

```
        qpDatabase.put("10-CS301-3",
        "Q1. Explain Socket Programming.\nQ2. Implement Transposition
Cipher.");
```

```
        setVisible(true);
    }
```

```
    private void startServer() {
        new Thread(() -> {
            try {
                serverSocket = new ServerSocket(8080);
                log("Server started on port 8080...");

                while (true) {
                    Socket client = serverSocket.accept();
                    log("Client connected");    handleClient(client);
                }
            } catch (Exception e) {
                log("Server Error: " + e.getMessage());
            }
        }).start();
    }
```

```
    private void handleClient(Socket client) {
        new Thread(() -> {
            try (
                BufferedReader in = new BufferedReader(
```

```

        new InputStreamReader(client.getInputStream()));
PrintWriter out = new PrintWriter(
    client.getOutputStream(), true)
    ) {
        // Receive request
        String cls = in.readLine();
        String courseCode = in.readLine();
in.readLine(); // course name (not used)
        String sem = in.readLine();

        String key = cls + "-" + courseCode + "-" + sem;
log("Request: " + key);

        String qp = qpDatabase.get(key);

        if (qp == null) {
out.println("ERROR");
            out.println("Question Paper not found");
return;
        }

        int transKey = 4; // fixed key
        String encryptedQP = transposeEncrypt(qp, transKey);

        out.println("OK");
out.println(transKey);
out.println(encryptedQP);

        log("Encrypted QP sent");

    } catch (Exception e) {
        log("Client Error: " + e.getMessage());
    }
}).start();
}

```

```

// Columnar Transposition Encryption    private
String transposeEncrypt(String text, int key) {
    text = text.replace("\n", " ");
    int len = text.length();
    int rows = (int) Math.ceil((double) len / key);

    char[][] matrix = new char[rows][key];
    int index = 0;

    for (int i = 0; i < rows; i++) {
    for (int j = 0; j < key; j++) {
    if (index < len)
        matrix[i][j] = text.charAt(index++);
    else
        matrix[i][j] = 'X'; // padding
    }
    }

    StringBuilder encrypted = new StringBuilder();
    for (int j = 0; j < key; j++) {        for (int i = 0; i <
rows; i++) {
        encrypted.append(matrix[i][j]);
    }
    }
    return encrypted.toString();
}

private void log(String msg) {
    logArea.append(msg + "\n");
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(ExamServerGUI::new);
}
}

```

```
import java.awt.*;
import java.io.*; import
java.net.*; import
java.nio.file.*;
import javax.swing.*;

public class ExamClientGUI extends JFrame {

    private JTextField classField, courseCodeField, semField, keyField;
    private JTextArea qpArea;    private String encryptedQP;

    public ExamClientGUI() {
        setTitle("Exam Client - Transposition Cipher");
        setSize(500, 500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        JPanel inputPanel = new JPanel(new GridLayout(5, 2));

        inputPanel.add(new JLabel("Class:"));
        classField = new JTextField("10");
        inputPanel.add(classField);

        inputPanel.add(new JLabel("Course Code:"));
        courseCodeField = new JTextField("CS101");
        inputPanel.add(courseCodeField);

        inputPanel.add(new JLabel("Semester:"));
        semField = new JTextField("1");
        inputPanel.add(semField);
```

```
inputPanel.add(new JLabel("Key:"));  
keyField = new JTextField();  
inputPanel.add(keyField);
```

```
 JButton requestBtn = new JButton("Request QP");  
requestBtn.addActionListener(e -> requestQP());  
inputPanel.add(requestBtn);
```

```
 JButton decryptBtn = new JButton("Decrypt & Download");  
decryptBtn.addActionListener(e -> decryptQP());  
inputPanel.add(decryptBtn);
```

```
add(inputPanel, BorderLayout.NORTH);
```

```
qpArea = new JTextArea();  
qpArea.setEditable(false);  
add(new JScrollPane(qpArea), BorderLayout.CENTER);
```

```
setVisible(true);  
}
```

```
private void requestQP() {  
    try {  
        Socket socket = new Socket("localhost", 8080);  
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);  
        BufferedReader in = new BufferedReader(  
            new InputStreamReader(socket.getInputStream()))  
        ) {  
            out.println(classField.getText());  
            out.println(courseCodeField.getText());  
            out.println("Dummy");          out.println(semField.getText());  
  
            String status = in.readLine();  
            if ("ERROR".equals(status)) {  
                qpArea.setText(in.readLine());  
            }  
            return;  
        }  
    }  
}
```



```

        int key = Integer.parseInt(in.readLine());
        encryptedQP = in.readLine();

        keyField.setText(String.valueOf(key));
        qpArea.setText("Encrypted QP:\n\n" + encryptedQP);

    } catch (Exception e) {
        qpArea.setText("Connection Error: " + e.getMessage());
    }
}

private void decryptQP() {
    try {
        int key = Integer.parseInt(keyField.getText());
        String decrypted = transposeDecrypt(encryptedQP, key);

        qpArea.setText("DECRYPTED QP:\n\n" + decrypted);

        Files.write(Paths.get("QuestionPaper.txt"), decrypted.getBytes());
        JOptionPane.showMessageDialog(this, "QP Downloaded");

    } catch (Exception e) {
        qpArea.setText("Decryption Error");
    }
}

// Columnar Transposition Decryption    private
String transposeDecrypt(String text, int key) {
    int len = text.length();
    int rows = len / key;

    char[][] matrix = new char[rows][key];
    int index = 0;

    for (int j = 0; j < key; j++) {
        for (int i = 0; i < rows; i++) {

```

```

        matrix[i][j] = text.charAt(index++);
    }
}

StringBuilder decrypted = new StringBuilder();
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < key; j++) {
        decrypted.append(matrix[i][j]);
    }
}
return decrypted.toString().replace("X", "");
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(ExamClientGUI::new);
}
}

```

TranspositionServer.java

```

import java.io.*;
import java.net.*;

public class TranspositionServer {

    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(8080);
            System.out.println("Server started on port 8080");

            Socket client = serverSocket.accept();
            System.out.println("Client connected");

            BufferedReader in = new BufferedReader(

```

```
        new InputStreamReader(client.getInputStream()));  
PrintWriter out = new PrintWriter(  
client.getOutputStream(), true);
```

```
String request = in.readLine();
```

```
String questionPaper = "HELLO WORLD";  
int key = 4;
```

```
String encryptedText = encrypt(questionPaper, key);
```

```
out.println(key);  
out.println(encryptedText);
```

```
System.out.println("Encrypted text sent: " + encryptedText);
```

```
client.close();  
serverSocket.close();
```

```
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

```
private static String encrypt(String text, int key) {  
int len = text.length();  
int rows = (int) Math.ceil((double) len / key);  
  
char[][] matrix = new char[rows][key];  
int index = 0;
```

```

        for (int i = 0; i < rows; i++) {
    for (int j = 0; j < key; j++) {
    if (index < len)
        matrix[i][j] = text.charAt(index++);
    else
        matrix[i][j] = 'X';
    }
    }

    StringBuilder encrypted = new StringBuilder();
    for (int j = 0; j < key; j++) {
        for (int i = 0; i <
rows; i++) {
            encrypted.append(matrix[i][j]);
        }
    }
    return encrypted.toString();
}
}

```

Output:



